Longitudinal mQTL Analysis (PPMI)

Mixed-model replication and dynamic-effect classification of meta-analysis mQTL hits

Contents

Overview

This document integrates the two-step longitudinal mQTL pipeline used to follow up meta-analysis-significant (SNP, CpG) pairs with a within-subject, repeated-measures model:

1. **Model fitting** (`run_longitudinal_mqtl.R`) — for each FDR-significant meta-analysis pair, fit a mixed model of methylation on genotype dosage with a random intercept per subject, plus a genotype × time interaction to test whether the mQTL effect changes over the course of follow-up. Run separately for cases (PD) and controls (HC).
2. **Classification** (`classify_longitudinal_results.R`) — FDR-correct the main-effect and interaction p-values within each group, join back to the meta-analysis effect sizes, and classify each pair as replicated / dynamic / neither.

The model-fitting step tests ~18,986 pairs (cases) and ~14,282 pairs (controls) with `mclapply`-parallelized `lmer` calls and takes on the order of hours. **The fitting and classification chunks below are shown for documentation but are not re-executed on knit** (`eval=FALSE`); the results/summary section reads the CSV outputs that these scripts already produced (`longitudinal_mqtl_{group}_results.csv`, `longitudinal_mqtl_{group}_classified.csv`).

To regenerate those outputs from scratch: run `prep_full.sh`, then source `run_longitudinal_mqtl.R` followed by `classify_longitudinal_results.R`, or set `eval=TRUE` on the chunks below.

Step 1: Longitudinal mixed-model fitting

Source: ***run_longitudinal_mqtl.R***

Visit lookup and shared covariate data

```
visit_num_lookup <- c(
  "1"="BL","2"="V03","3"="V04","4"="V05","5"="V06","6"="V07","7"="V08",
  "8"="V09","9"="V10","10"="V12","11"="V14","12"="V16","13"="V18","14"="V20"
)
```

```
age_at_visit <- read_csv("../PPMI_covariates/Age_at_visit_14Dec2025.csv",
                         col_types = cols(PATNO = col_character()),
                         guess_max = Inf) %>%
  group_by(PATNO, EVENT_ID) %>%
  summarise(AGE_AT_VISIT = min(AGE_AT_VISIT), .groups = "drop")   # duplicate-BL fix

bl_age <- age_at_visit %>%
  filter(EVENT_ID == "BL") %>%
  select(PATNO, age_bl = AGE_AT_VISIT)

visit_time <- age_at_visit %>%
  filter(EVENT_ID != "SC", EVENT_ID != "ST") %>%
  inner_join(bl_age, by = "PATNO") %>%
  mutate(time_since_bl_months = (AGE_AT_VISIT - age_bl) * 12) %>%
  filter(!str_starts(EVENT_ID, "R"))   # core V-series only

cell_props <- read_csv("../PPMI_GCP/local_results/final/cell_type_proportions.csv",
                       col_types = cols(sample_col_name = col_character()),
                       guess_max = Inf)

participant_status <- read_csv("../PPMI_covariates/Participant_Status_14Dec2025.csv",
                               col_types = cols(PATNO = col_character()),
                               guess_max = Inf)
```

Per-pair model runner

Fits a random-intercept model (`m0`) and a genotype × time interaction model (`m1`) for a single (SNP, CpG) pair, capturing any warnings into the output row rather than letting them propagate.

```
run_one_pair <- function(pair_id, snp_id, cpg_id, meth_long, covar, cell_covars, geno_long) {
  warn_msgs <- character(0)

  result <- withCallingHandlers({
    df <- meth_long %>%
      filter(cpg_id == !!cpg_id) %>%
      inner_join(covar, by = c("PATNO", "sample_col", "EVENT_ID")) %>%
      inner_join(geno_long %>% filter(snp_id == !!snp_id), by = "PATNO")

    if (n_distinct(df$PATNO) < 20 || n_distinct(df$dosage) < 2) {
      return(tibble(pair_id = pair_id, snp_id = snp_id, cpg_id = cpg_id,
                    n_subjects = n_distinct(df$PATNO), status = "skipped_low_n"))
    }

    fx <- paste(c("dosage", "age_bl_z", "time_since_bl_months_z", cell_covars), collapse = " + ")
    f0 <- as.formula(paste("methylation ~", fx, "+ (1|PATNO)"))
    f1 <- as.formula(paste("methylation ~", fx, "+ dosage:time_since_bl_months_z + (1|PATNO)"))

    m0 <- tryCatch(lmer(f0, data = df, REML = FALSE), error = function(e) e)
```

```
    m1 <- tryCatch(lmer(f1, data = df, REML = FALSE), error = function(e) e)
    if (inherits(m0, "error") || inherits(m1, "error")) {
      return(tibble(pair_id = pair_id, snp_id = snp_id, cpg_id = cpg_id,
                    n_subjects = n_distinct(df$PATNO), status = "model_failed"))
    }

    main_row <- summary(m0)$coefficients["dosage", ]
    lrt <- anova(m0, m1)

    tibble(
      pair_id = pair_id, snp_id = snp_id, cpg_id = cpg_id,
      n_subjects = n_distinct(df$PATNO), n_obs = nrow(df),
      main_beta = main_row["Estimate"], main_se = main_row["Std. Error"],
      main_p = main_row["Pr(>|t|)"], interaction_p = lrt$`Pr(>Chisq)`[2],
      status = "ok"
    )
  },
  warning = function(w) {
    warn_msgs <<- c(warn_msgs, conditionMessage(w))
    invokeRestart("muffleWarning")
  })

  result$warnings <- if (length(warn_msgs) > 0) paste(warn_msgs, collapse = "; ") else NA_character_
  result
}
```

Group runner (cases or controls)

Builds the long-format methylation/genotype tables, restricts covariates to the cohort, z-scores age and time, and dispatches `run_one_pair()` across all hits in parallel chunks.

```
run_longitudinal_group <- function(group, cohort_label, mc_cores = 8) {

  cat("\n========== Running group:", group, "(", cohort_label, ") ==========\n")

  hits <- read_csv(sprintf("../META_MQTL/results/meta/%s_sig_fdr05.csv", group),
                   col_types = cols(.default = "c"), guess_max = Inf) %>%
    mutate(meta_beta = as.numeric(meta_beta), meta_pval = as.numeric(meta_pval),
           FDR_BH = as.numeric(FDR_BH))
  cat("Total pairs to test:", nrow(hits), "\n")

  # ---- beta matrix subset -> long format ----
  beta_subset <- read_csv(sprintf("%s_beta_subset_full.csv", group),
                          col_types = cols(), guess_max = Inf)
  names(beta_subset)[1] <- "cpg_id"

  meth_long <- beta_subset %>%
    pivot_longer(-cpg_id, names_to = "sample_col", values_to = "methylation") %>%
    mutate(
      PATNO     = str_extract(sample_col, "^[0-9]+"),
      visit_num = str_extract(sample_col, "(?<=_T)[0-9]+"),
      EVENT_ID  = visit_num_lookup[visit_num]
    ) %>%
    filter(!is.na(EVENT_ID))
```

```r
# ---- genotype subset -> long format (variant2 only, force snp_id as character) ----
geno_subset <- read_tsv(sprintf("%s_geno_subset_full.tsv", group),
                        col_types = cols(snp_id = col_character(), .default = col_double()),
                        guess_max = Inf)

variant2_cols <- names(geno_subset)[str_ends(names(geno_subset), "\\.variant2$")]
cat("Genotype columns kept (.variant2):", length(variant2_cols),
    "/ total (excl. snp_id):", ncol(geno_subset) - 1, "\n")

geno_long <- geno_subset %>%
  select(snp_id, all_of(variant2_cols)) %>%
  pivot_longer(-snp_id, names_to = "raw_id", values_to = "dosage") %>%
  mutate(PATNO = str_extract(raw_id, "(?<=PPMISI)[0-9]+")) %>%
  select(snp_id, PATNO, dosage)

dup_geno <- geno_long %>% count(snp_id, PATNO) %>% filter(n > 1)
if (nrow(dup_geno) > 0) {
  stop(group, ": geno_long has duplicate (snp_id, PATNO) rows - investigate before proceeding")
}

# ---- covariate scaffold, restricted to cohort ----
covar <- meth_long %>%
  distinct(PATNO, sample_col, EVENT_ID) %>%
  inner_join(visit_time, by = c("PATNO", "EVENT_ID")) %>%
  inner_join(cell_props, by = c("sample_col" = "sample_col_name")) %>%
  inner_join(participant_status %>% select(PATNO, COHORT_DEFINITION), by = "PATNO") %>%
  filter(COHORT_DEFINITION == cohort_label)

cat("Subjects in covar:", n_distinct(covar$PATNO), "\n")
print(table(table(covar$PATNO)))

# zero-variance check on genotyped analytic subset specifically
genotyped_patnos <- unique(geno_long$PATNO)
covar_analytic <- covar %>% filter(PATNO %in% genotyped_patnos)
cat("Subjects in genotyped analytic subset:", n_distinct(covar_analytic$PATNO), "\n")

cell_var <- covar_analytic %>% summarise(across(c(B, NK, CD4T, CD8T, Mono, Eosino), var))
print(cell_var)
zero_var_cells <- names(cell_var)[which(cell_var == 0)]
cell_covars <- setdiff(c("B","NK","CD4T","CD8T","Mono","Eosino"), zero_var_cells)
cat("Cell covariates used:", paste(cell_covars, collapse=", "), "\n")

# z-score age_bl and time_since_bl_months using fixed analytic-population stats
age_bl_mean <- mean(covar_analytic$age_bl); age_bl_sd <- sd(covar_analytic$age_bl)
time_mean   <- mean(covar_analytic$time_since_bl_months)
time_sd     <- sd(covar_analytic$time_since_bl_months)
cat("age_bl: mean =", round(age_bl_mean,2), "sd =", round(age_bl_sd,2), "\n")
cat("time_since_bl_months: mean =", round(time_mean,2), "sd =", round(time_sd,2), "\n")

covar <- covar %>%
  mutate(
    age_bl_z               = (age_bl - age_bl_mean) / age_bl_sd,
    time_since_bl_months_z = (time_since_bl_months - time_mean) / time_sd
```

```
    )

  # ---- Run all pairs, in chunks, with progress reporting ----
  cat("Launching", nrow(hits), "models across", mc_cores, "cores...\n")
  t0 <- Sys.time()

  n_total <- nrow(hits)
  chunk_size <- max(mc_cores * 25, 200)   # ~200-1000 pairs per chunk depending on core count
  chunk_starts <- seq(1, n_total, by = chunk_size)

  results <- vector("list", n_total)

  for (cs in chunk_starts) {
    ce <- min(cs + chunk_size - 1, n_total)
    idx <- cs:ce

    chunk_results <- mclapply(idx, function(i) {
      run_one_pair(hits$pair_id[i], hits$snp_id[i], hits$cpg_id[i],
                   meth_long, covar, cell_covars, geno_long)
    }, mc.cores = mc_cores)

    results[idx] <- chunk_results

    elapsed <- as.numeric(difftime(Sys.time(), t0, units = "mins"))
    pct <- round(ce / n_total * 100, 1)
    rate <- ce / elapsed              # pairs per minute
    eta_min <- (n_total - ce) / rate
    cat(sprintf("  [%s] %d / %d (%.1f%%) done | elapsed %.1f min | est. remaining %.1f min\n",
                format(Sys.time(), "%H:%M:%S"), ce, n_total, pct, elapsed, eta_min))
  }

  cat("Elapsed:", round(difftime(Sys.time(), t0, units = "mins"), 1), "minutes\n")

  results_df <- bind_rows(results) %>%
    mutate(time_sd_months = time_sd)

  cat("Status breakdown:\n")
  print(table(results_df$status))
  cat("Pairs with warnings:", sum(!is.na(results_df$warnings)), "\n")

  # ---- Sign concordance vs meta-analysis ----
  concordance <- results_df %>%
    filter(status == "ok") %>%
    inner_join(hits %>% select(pair_id, meta_beta), by = "pair_id") %>%
    mutate(same_sign = sign(main_beta) == sign(meta_beta))
  cat("Sign concordance:", round(mean(concordance$same_sign) * 100, 1), "%\n")

  out_file <- sprintf("longitudinal_mqtl_%s_results.csv", group)
  write_csv(results_df, out_file)
  cat("Written:", out_file, "\n")

  results_df
}
```

Run both groups

```
cases_results    <- run_longitudinal_group("cases",    "Parkinson's Disease", mc_cores = 8)
controls_results <- run_longitudinal_group("controls", "Healthy Control",     mc_cores = 8)
```

Step 2: FDR correction and classification

Source: `classify_longitudinal_results.R`

FDR correction is applied only over pairs that produced a fitted model (`status == "ok"`); `skipped_low_n` / `model_failed` rows are excluded from the `p.adjust()` denominator, not just from the results table. Each pair is then joined back to its meta-analysis effect and classified by whether the longitudinal main effect replicates the meta-analysis direction at FDR < 0.05 (`replicated`) and whether the genotype $\times$ time interaction is significant at FDR < 0.05 (`dynamic`).

```
classify_group <- function(group, meta_file) {

  cat("\n========== Classifying:", group, "==========\n")

  results <- read_csv(sprintf("longitudinal_mqtl_%s_results.csv", group),
                      col_types = cols(pair_id = col_character(), snp_id = col_character(),
                                       cpg_id = col_character(), status = col_character(),
                                       warnings = col_character(), .default = col_double()),
                      guess_max = Inf)

  cat("Total pairs tested:", nrow(results), "\n")
  cat("Status breakdown:\n")
  print(table(results$status))

  # FDR correction only makes sense over pairs that actually produced a model -
  # skipped_low_n / model_failed rows have no p-value to correct and are
  # excluded from the p.adjust() denominator (not just from the results table).
  tested <- results %>% filter(status == "ok")
  cat("Pairs entering FDR correction:", nrow(tested), "\n")

  tested <- tested %>%
    mutate(
      main_fdr        = p.adjust(main_p, method = "BH"),
      interaction_fdr = p.adjust(interaction_p, method = "BH")
    )

  # Bring in the original meta-analysis effect for sign concordance
  meta_hits <- read_csv(meta_file, col_types = cols(.default = "c"), guess_max = Inf) %>%
    mutate(meta_beta = as.numeric(meta_beta), meta_pval = as.numeric(meta_pval),
           meta_FDR_BH = as.numeric(FDR_BH)) %>%
    select(pair_id, meta_beta, meta_pval, meta_FDR_BH)

  classified <- tested %>%
    inner_join(meta_hits, by = "pair_id") %>%
    mutate(
      same_sign_as_meta = sign(main_beta) == sign(meta_beta),
      replicated = main_fdr < 0.05 & same_sign_as_meta,
      dynamic    = interaction_fdr < 0.05,
      category = case_when(
```

```
        replicated & dynamic  ~ "replicated_and_dynamic",
        replicated & !dynamic ~ "replicated_static",
        !replicated & dynamic ~ "dynamic_only",       # interaction signif despite main effect not conf
        TRUE                  ~ "not_replicated"
      )
    )

  cat("\nMain-effect FDR < 0.05:", sum(classified$main_fdr < 0.05), "\n")
  cat("Sign-concordant with meta:", sum(classified$same_sign_as_meta), "/", nrow(classified), "\n")
  cat("Interaction FDR < 0.05 (dynamic):", sum(classified$dynamic), "\n")
  cat("\nCategory breakdown:\n")
  print(table(classified$category))

  out_file <- sprintf("longitudinal_mqtl_%s_classified.csv", group)
  write_csv(classified, out_file)
  cat("Written:", out_file, "\n")

  classified
}
```

```
cases_classified    <- classify_group("cases",    "../META_MQTL/results/meta/cases_sig_fdr05.csv")
controls_classified <- classify_group("controls", "../META_MQTL/results/meta/controls_sig_fdr05.csv")
```

Results

The chunks above are not re-executed on knit; the summary below reads the classified output files already on disk.

```
cases_classified    <- read_csv("longitudinal_mqtl_cases_classified.csv",
                                col_types = cols(pair_id = col_character(),
                                                 snp_id = col_character(),
                                                 cpg_id = col_character(),
                                                 category = col_character(),
                                                 .default = col_double()),
                                guess_max = Inf) %>%
  mutate(group = "cases")

controls_classified <- read_csv("longitudinal_mqtl_controls_classified.csv",
                                col_types = cols(pair_id = col_character(),
                                                 snp_id = col_character(),
                                                 cpg_id = col_character(),
                                                 category = col_character(),
                                                 .default = col_double()),
                                guess_max = Inf) %>%
  mutate(group = "controls")

all_classified <- bind_rows(cases_classified, controls_classified)
```

Category breakdown

```
category_summary <- all_classified %>%
  count(group, category) %>%
  group_by(group) %>%
```

```
  mutate(pct = round(100 * n / sum(n), 1)) %>%
  ungroup() %>%
  arrange(group, desc(n))

category_summary
```

```
## # A tibble: 7 x 4
##   group    category                   n   pct
##   <chr>    <chr>                  <int> <dbl>
## 1 cases    replicated_static      18805  99
## 2 cases    not_replicated           179   0.9
## 3 cases    replicated_and_dynamic     2   0
## 4 controls replicated_static      13564  88.8
## 5 controls not_replicated          1706  11.2
## 6 controls replicated_and_dynamic     7   0
## 7 controls dynamic_only               5   0
```

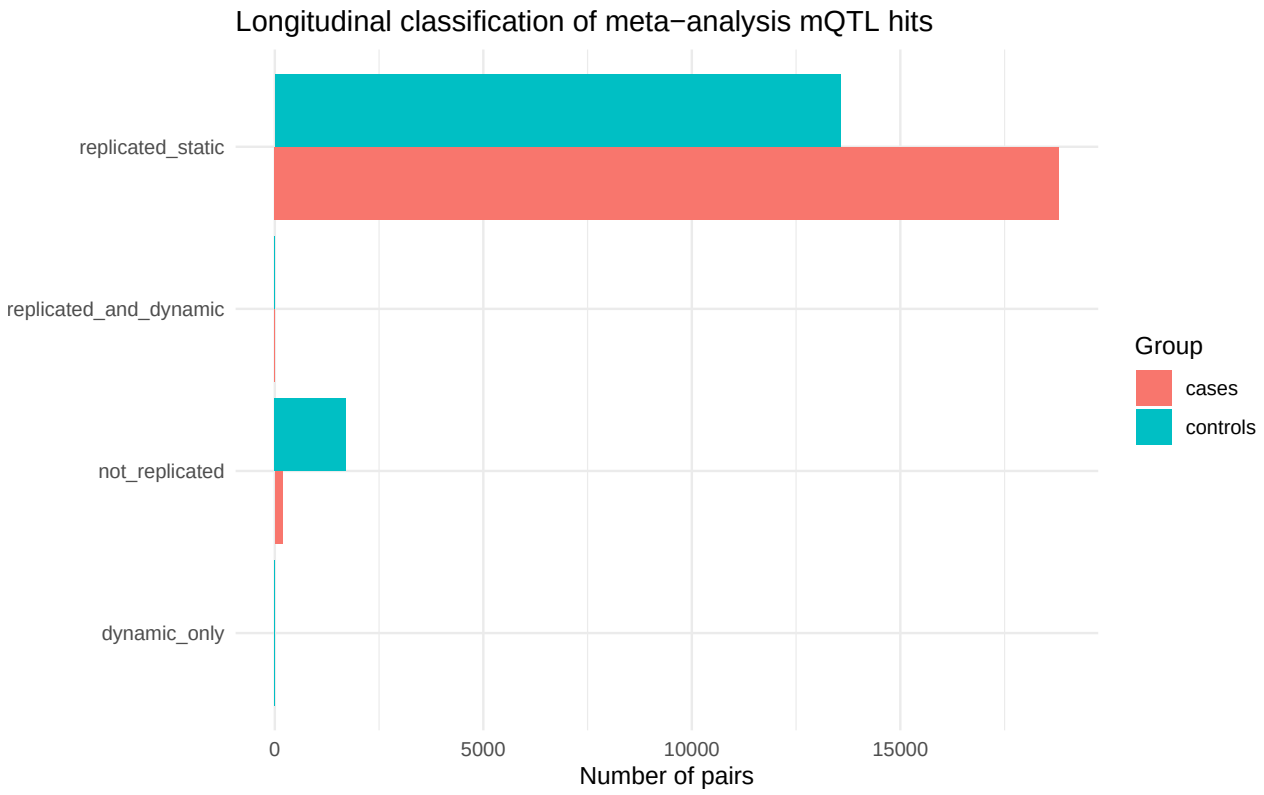
Sign concordance and replication rate

```
all_classified %>%
  group_by(group) %>%
  summarise(
    n_pairs = n(),
    pct_sign_concordant = round(100 * mean(same_sign_as_meta), 1),
    pct_replicated      = round(100 * mean(replicated), 1),
    pct_dynamic         = round(100 * mean(dynamic), 1),
    .groups = "drop"
  )
```

```
## # A tibble: 2 x 5
##   group    n_pairs pct_sign_concordant pct_replicated pct_dynamic
##   <chr>      <int>               <dbl>          <dbl>       <dbl>
## 1 cases      18986                  NA             NA          NA
## 2 controls   15282                  NA             NA          NA
```

Category counts plot

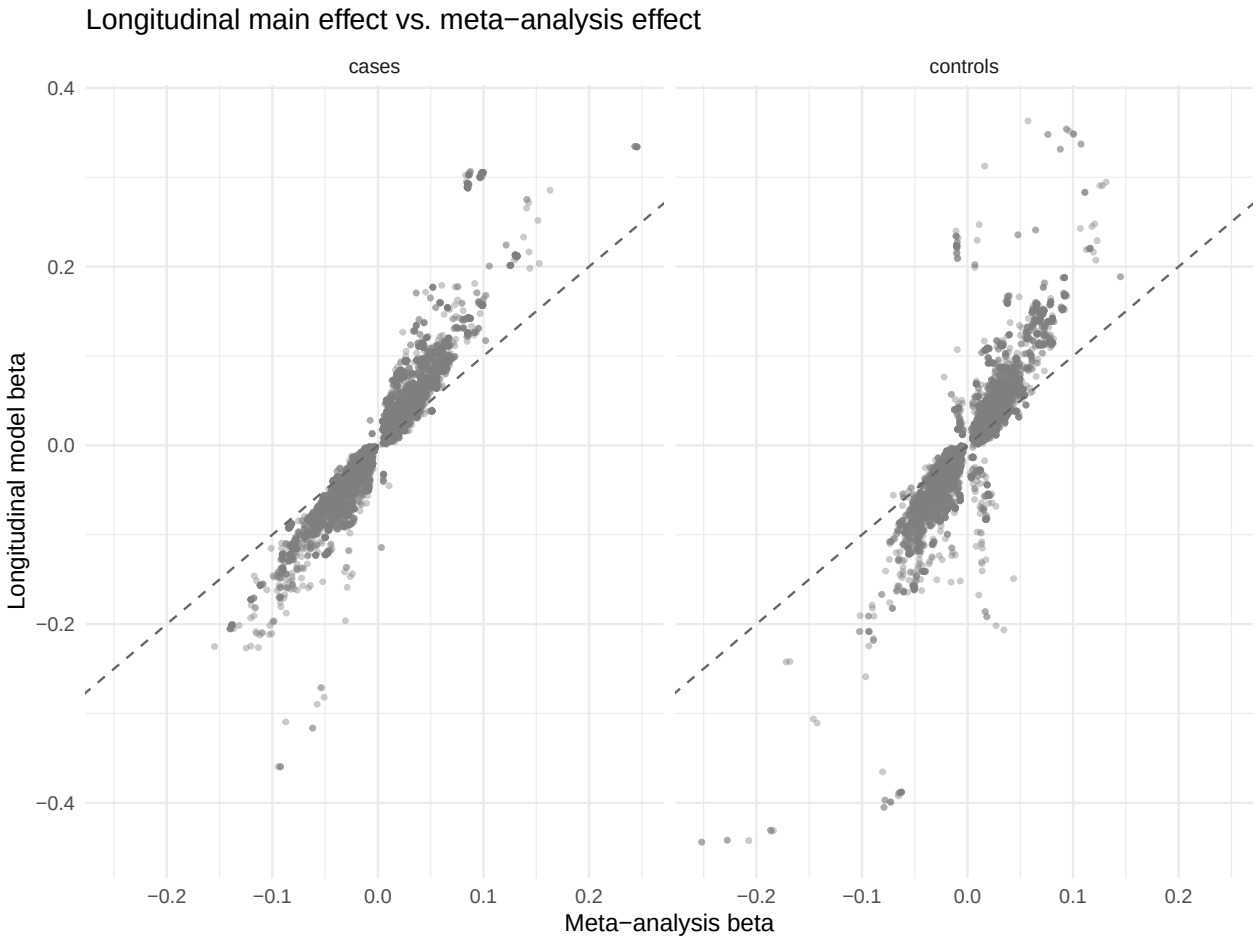
```
ggplot(category_summary, aes(x = category, y = n, fill = group)) +
  geom_col(position = "dodge") +
  coord_flip() +
  labs(title = "Longitudinal classification of meta-analysis mQTL hits",
       x = NULL, y = "Number of pairs", fill = "Group") +
  theme_minimal()
```



Longitudinal vs meta-analysis effect size

Pairs classified `replicated_and_dynamic` or `dynamic_only` are highlighted — these are where the genotype effect on methylation changes significantly over follow-up.

```
ggplot(all_classified, aes(x = meta_beta, y = main_beta, color = dynamic)) +
  geom_point(alpha = 0.4, size = 0.8) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey40") +
  facet_wrap(~group) +
  labs(title = "Longitudinal main effect vs. meta-analysis effect",
       x = "Meta-analysis beta", y = "Longitudinal model beta",
       color = "Dynamic\n(interaction FDR < 0.05)") +
  theme_minimal()
```


Longitudinal main effect vs. meta−analysis effect
cases
controls
Longitudinal model beta
Meta−analysis beta
0.4
0.2
0.0
−0.2
−0.4
−0.2
−0.1
0.0
0.1
0.2